



**INSTITUTO
FEDERAL**
Santa Catarina

Estratégias de Ramificação

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#384254	oxford blue
acento escuro	#9599AE	santas gray
cor principal	#E74146	cinnabar
acento claro	#E2AC5B	equator
tom claro	#F8FAF9	snow drift
	#252A34	

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono

Gerenciamento de Repositórios

histórico

■ décadas de 1950-1960

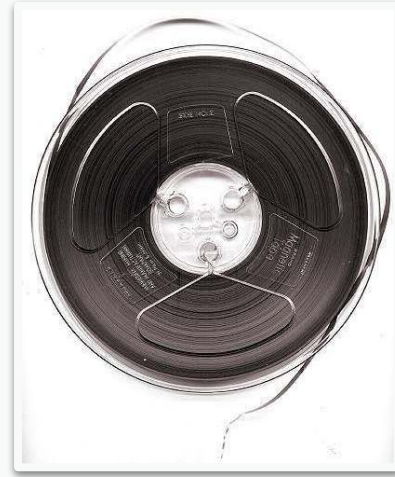
- cartões perfurados
 - cada cartão representa uma linha
 - um conjunto de cartões para um programa
 - guardados em uma gaveta ou caixa
- fazer check-out era retirar o cartão da gaveta
- posse física do programa
- ao fim do trabalho, o cartão retorna para a gaveta
- **ciclo de tempo** enquanto houver posse do cartão



Gerenciamento de Repositórios

histórico

- décadas de 1950-1960: cartões perfurados
- **década de 1970**
 - fitas magnéticas
 - grande quantidade de módulos
 - fáceis de duplicar
 - fita mestra no armário principal
 - módulo copiado para uma fita de trabalho
 - quadro de verificação com alfinetes coloridos
 - alfinete ao lado do nome do módulo indicava que ele estava sendo editado
 - **ciclo de tempo** enquanto o alfinete estiver no quadro



Gerenciamento de Repositórios

histórico

- décadas de 1950-1960: cartões perfurados
- década de 1970: fitas magnéticas
- **década de 1980**
 - discos e SCCS
 - emulavam o quadro de verificação
 - *lock pessimista* bloqueava um módulo no disco
 - SCCS -> RCS -> CVS
 - discos são mais convenientes do que fitas
 - possibilitavam módulos menores
 - reduziram o ciclo de tempo



Gerenciamento de Repositórios

histórico

- décadas de 1950-1960: cartões perfurados
- década de 1970: fitas magnéticas
- década de 1980: discos e SCCS
- **anos 2000**
 - subversion (SVN)
 - evolução do CVS
 - *lock otimista* não bloqueava o módulo
 - check-outs ao mesmo tempo
 - alterações rastreadas e mescladas automaticamente
 - conflitos resolvidos antes do check-in



Gerenciamento de Repositórios

histórico

- décadas de 1950-1960: cartões perfurados
- década de 1970: fitas magnéticas
- década de 1980: discos e SCCS
- anos 2000: subversion
- **2005**
 - git
 - criado para manter o kernel linux
 - velocidade, segurança, trabalho distribuído
 - cada clone contém todo o histórico
 - ramificações e mesclagens eficientes



Gerenciamento de Repositórios

histórico

- décadas de 1950-1960: cartões perfurados
- década de 1970: fitas magnéticas
- década de 1980: discos e SCCS
- anos 2000: subversion
- 2005: git
- **2010 em diante**
 - plataformas colaborativas
 - github, gitlab, bitbucket
 - pull requests / merge requests
 - issues, wiki, actions (ci/cd)



Gerenciamento de Repositórios

sistema de controle de versão

- o que controlar (*commit*)
 - tudo necessário para construção do projeto
 - código
 - scripts
 - migrações, schemas
 - configurações de IDEs
- o que não controlar
 - o que pode ser construído
 - gem, jar, image, modules

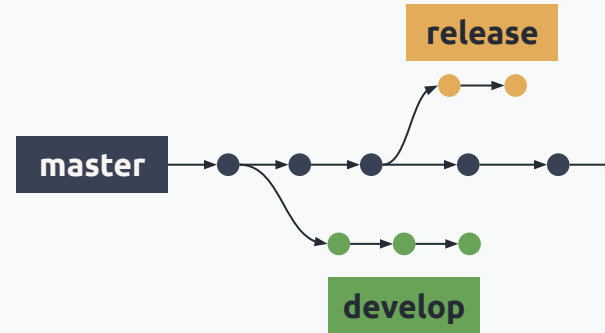
organização dos repositórios

- multi-repo
 - cada projeto no seu repositório
- mono-repo
 - um único repositório para todos os projetos
 - administração mais simples
 - refatorações globais

Gerenciamento de Repositórios

boas práticas

- commits simples e lançáveis
- ramos atrasam a integração
- ramos de vida curta são mais simples de mesclar
- muitos ramos geram mais burocracia
- estratégias devem ser combinadas



Gerenciamento de Repositórios

pull request

- pedido de mudança no código
- processo
 - criar branch
 - fazer *commits* com as alterações do código
 - abrir *pull request*
- GitHub permite
 - ver o que mudou (*diff*)
 - comentar linha a linha
 - pedir revisão (*review*)
 - rodar *checks/CI*
 - mesclar o código

importância

- evita *push* direto no ramo principal
- permite revisão
- cria rastreabilidade (quem mudou, por quê, quando, ligado a uma issue)
- integra com automação

Gerenciamento de Repositórios

estrutura do pull request

■ contexto

- onde ocorre o problema/necessidade
- qual parte do sistema é afetada

■ motivação

- qual problema resolve
- qual requisito cumpre
- link com *issue*/história

■ evidências

- passos de teste manual + resultado esperado
- *print* / gif / vídeo (UI)

- *logs* (*backend*)

- teste automatizado adicionado
- link do check/CI verde

■ riscos e impacto

- desempenho
- compatibilidade
- dados
- segurança
- comportamento em casos limite

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- colocar um rótulo antes de cada comentário
 - [clareza] [correção] [manutenção]
[segurança] [teste]
- usar o formato P-M-A
 - **problema** o que está errado/arriscado
 - **motivo** por que importa (bug, manutenção, segurança)
 - **alternativa** sugestão concreta

exemplos

[correção]

Problema: `if (valor) falha quando valor = 0.`

Motivo: 0 é valor válido (doação) e o sistema vai rejeitar indevidamente.

Alternativa: validar `valor != null` e checar `valor >= 0.`

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- **clareza**
 - PR tem descrição (o quê/por quê/como testar)?
 - PR é pequeno e focado?
 - nome de branch e título do PR fazem sentido?

exemplos

[clareza] A descrição não explica o cenário do bug. Pode incluir 'como reproduzir' e o resultado esperado?

[clareza] Esse PR mistura ajuste de CSS com correção de regra de negócio. Sugiro separar em dois PRs para facilitar revisão e rollback.

ruim (opinião): Não gostei desse título.

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- clareza
- **correção**
 - lida com casos óbvios? (vazio, 0, null, lista vazia)
 - evita comportamento inesperado?
 - a mudança bate com o que a *issue* pede?

exemplos

[correção] `if(value) falha para 0. Melhor value != null.`

[correção] Se `quantidade = 0`, esse cálculo explode. Qual é o comportamento esperado?

[correção] E se a data de compra for futura? Precisamos bloquear ou tratar?

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- clareza
- correção
- **manutenibilidade**
 - nomes claros (funções/variáveis/arquivos)
 - duplicação de código
 - complexidade
 - coesão

exemplos

[manutenção] Esse trecho aparece em 3 lugares. Sugiro extrair `validatePatrimonio()` para evitar divergência futura.

[manutenção] A função `process()` está fazendo validação + persistência + formatação. Melhor separar em 2 funções para facilitar testes.

ruim (preferência): Prefiro nomes em inglês.

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- clareza
- correção
- manutenibilidade
- **segurança e qualidade**
 - validação de entrada
 - tratamento de erro
 - LGPD (dados pessoais: CPF, telefone, e-mail)
 - permissões

exemplos

[segurança] Evitar logar CPF completo no console/log do servidor. Sugiro mascarar ou remover.

[segurança] Se a API retornar 403/401, a UI precisa tratar e mostrar mensagem (evita tela quebrada).

[segurança] Validação está só no frontend. O backend também precisa validar (senão qualquer um burla).

Gerenciamento de Repositórios

code review

- **objetivo** reduzir risco antes de entrar na *main*
- clareza
- correção
- manutenibilidade
- segurança e qualidade
- **testabilidade e evidência**
 - tem "como testar" no PR?
 - tem teste automatizado ou evidência manual?
 - CI/checks rodaram?

exemplos

[teste] Pode incluir passos de teste com dados de exemplo e resultado esperado?

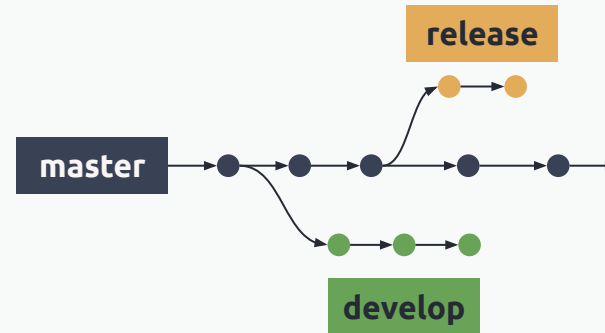
[teste] Esse bug merece um teste de regressão: caso "bem com 6 anos" não pode ficar negativo.

[teste] O PR altera regra de cálculo. Quais cenários você validou?

Estratégias de Ramificação

nomenclatura

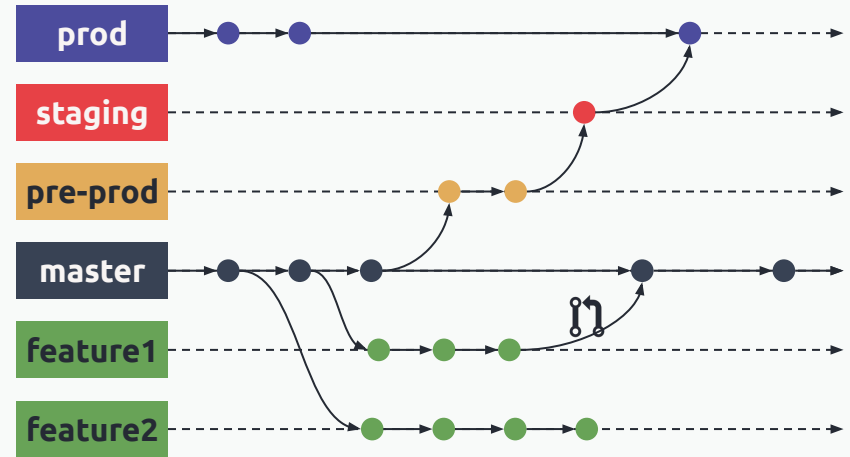
- temporários
 - ramos locais
- feature branches
 - implementam funcionalidades ou tarefas
- historical branches
 - master e develop
- environment branches
 - staging e production
- maintenance branches
 - release e hotfix



Estratégias de Ramificação

modelos de ramificação

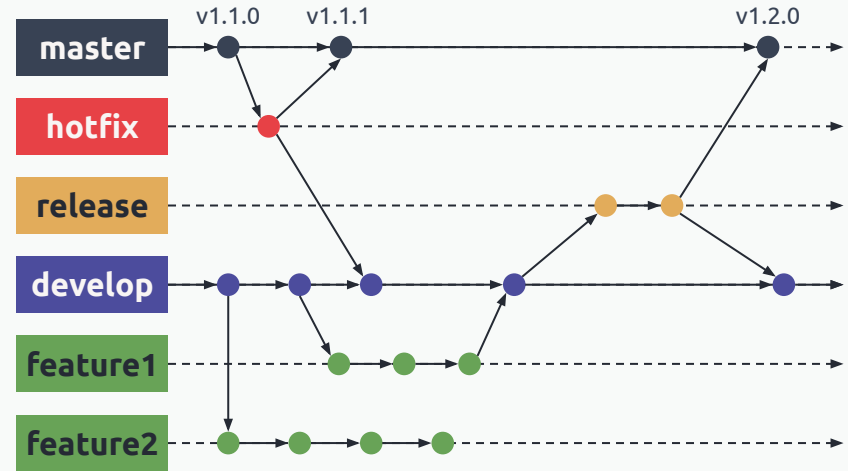
- *trunk-based development*
 - evita ramificações
- *feature branch workflow*
 - ramificações para cada funcionalidade
- *github flow*
 - feature branches + pull request
- *gitlab flow*
 - feature + env branches + pull request



Estratégias de Ramificação

modelos de ramificação

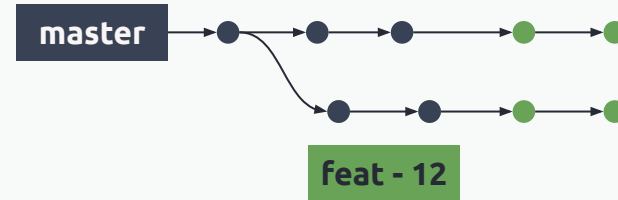
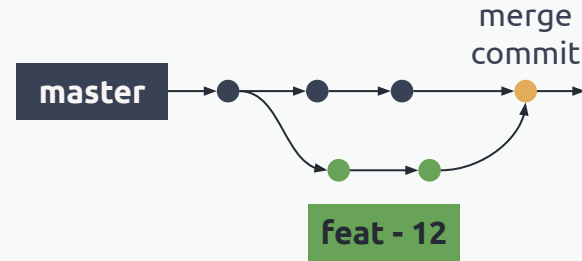
- trunk-based development
 - evita ramificações
- feature branch workflow
 - ramificações para cada funcionalidade
- github flow
 - feature branches + pull request
- gitlab flow
 - feature + env branches + pull request
- **git flow**
 - feature + maintenance + historical branches + pull request



Estratégias de Ramificação

sincronizando ramos

- mesclagem (*merge*)
 - cria um *commit* para a junção
 - o histórico depende dos ramos
- alteração da base (*rebase*)
 - altera a base do *commit*
 - mantém o histórico linear
 - permite a remoção do ramo





**INSTITUTO
FEDERAL**
Santa Catarina

Estratégias de Ramificação

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

